S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 07

Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Name: Chanchal Manwatkar
USN: CD25D002

Semester / Year: 4th SEM / 2nd YEAR
Academic Session: 2025-26

Date of Performance:

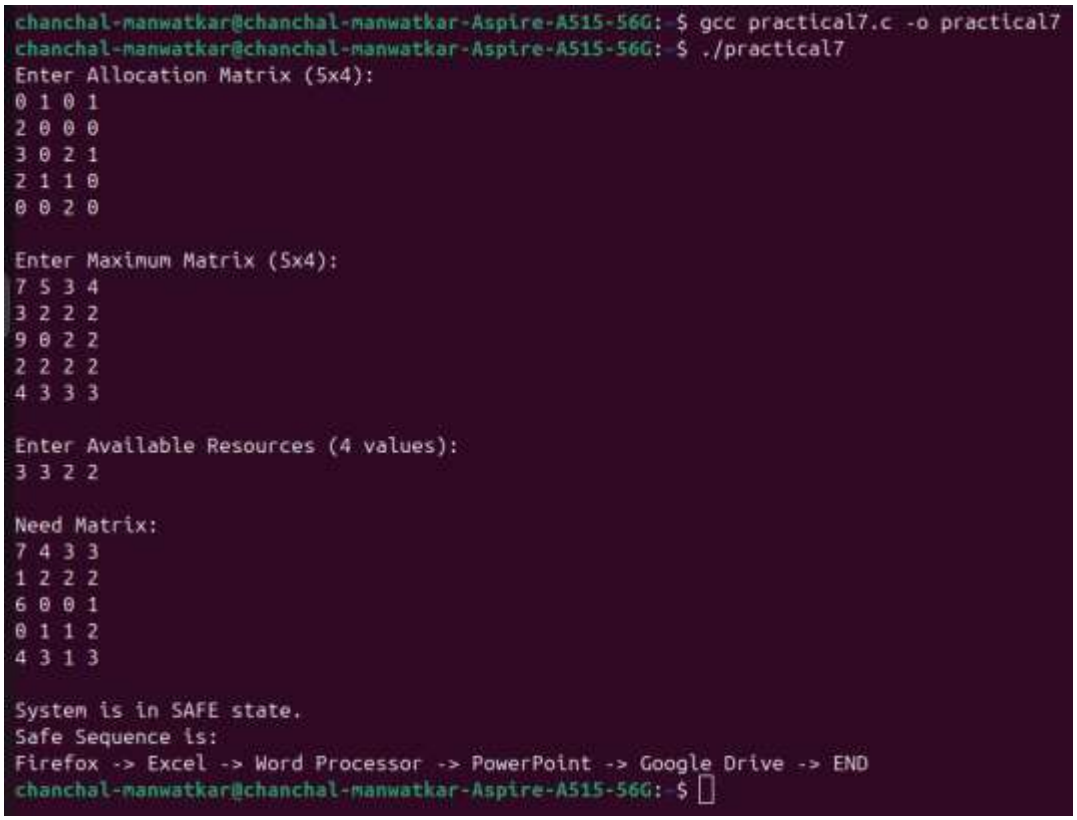
Date of Submission:

❖ CODE :

```
Firefox -> Excel -> Word Processor -> PowerPoint -> Google Drive -> END
chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G:~$ cat practical7.c
#include <stdio.h>
#include <stdbool.h>
int main() {
    int n = 5;  // Number of processes
    int m = 4;  // Number of resource types
    int allocation[5][4], max[5][4], need[5][4];
    int available[4];
    printf("Enter Allocation Matrix (5x4):\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &allocation[i][j]);
    printf("\nEnter Maximum Matrix (5x4):\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);
    printf("\nEnter Available Resources (4 values):\n");
    for(int i = 0; i < m; i++)
        scanf("%d", &available[i]);
    // Calculate Need matrix
    printf("\nNeed Matrix:\n");
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
    bool finish[5] = {false};
    int safeSeq[5];
    int work[4];
    for(int i = 0; i < m; i++)
        work[i] = available[i];
    int count = 0;
    while(count < n) {
        bool found = false;
        for(int i = 0; i < n; i++) {
            if(!finish[i]) {
                bool possible = true;
                for(int j = 0; j < m; j++) {
                    if(need[i][j] > work[j]) {
                        possible = false;
                        break;
                    }
```

```
                    break;
                }
            }

            if(possible) {
                for(int j = 0; j < m; j++)
                    work[j] += allocation[i][j];

                safeSeq[count++] = i;
                finish[i] = true;
                found = true;
            }
        }
    }

    if(!found) {
        printf("\nSystem is NOT in safe state (Deadlock possible).\n");
        return 0;
    }
}

printf("\nSystem is in SAFE state.\nSafe Sequence is:\n");

char *processNames[5] = {
    "Google Drive",
    "Firefox",
    "Word Processor",
    "Excel",
    "PowerPoint"
};

for(int i = 0; i < n; i++)
    printf("%s -> ", processNames[safeSeq[i]]);

printf("END\n");

return 0;
}
chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G:~$
```

❖ OUTPUT :


chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G: $ gcc practical7.c -o practical7
chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G: $ ./practical7
Enter Allocation Matrix (5x4):
0 1 0 1
2 0 0 0
3 0 2 1
2 1 1 0
0 0 2 0
Enter Maximum Matrix (5x4):
7 5 3 4
3 2 2 2
9 0 2 2
2 2 2 2
4 3 3 3
Enter Available Resources (4 values):
3 3 2 2
Need Matrix:
7 4 3 3
1 2 2 2
6 0 0 1
0 1 1 2
4 3 1 3
System is in SAFE state.
Safe Sequence is:
Firefox -> Excel -> Word Processor -> PowerPoint -> Google Drive -> END
chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G: $